⇒ Non-linearly Separable Prob^m

↳ We allow "error" ξ_i in classificat^n based on o/p of discriminant fun. $w^T x + b$.

↳ approximates the no. of misclassified points

↳ New objective fun. $\frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \xi_i$ (C = tradeoff hyperparameter b/w error & margin

s.t. $y_i(w x_i + b) \geq 1 - \xi_i$ & $\xi_i \geq 0$ ↳ slack variable.

large C = high penalty for errors

Regularizat^n Parameter

Dual Prob^m

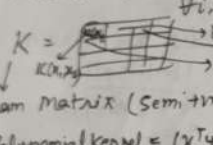
$\max W(\alpha) = \sum_i \alpha_i - \frac{1}{2}\sum_i \sum_j \alpha_i \alpha_j y_i y_j \, x_i \cdot x_j$

s.t. $C \geq \alpha_i \geq 0$

$\sum \alpha_i y_i = 0$

⇒ Every semi-positive definite symmetric funct^n is a kernel (Mercer's Theorem)

↳ $K(x_i, x_j) = K(x_j, x_i)$ $\forall i, j$

$K =$ 

Gram Matrix (Semi +ve definite symmetric)

(*) Polynomial Kernel $= (x^T y + 1)^d$ (d = degree)

(*) RBF Kernel $= \exp(-\|x - y\|^2 / 2\sigma^2)$

(*) $K(x, y) = \tanh(k x^T y + \theta)$. (~ = sigmoid fn)

(*) Non-Linear SVMs (Kernel Machines)

↳ Ex. XOR prob^m can't be solved by LSVM (Linear SVM)

↳ Idea: The original input space can be mapped to some higher dimensional feature space $\phi(x)$ where data is separable.

↳ N data points are separable in N-1 dim. space

↳ Comput^n is higher to transform low dimension to higher.

↳ Kernel tricks solves this issue.

↳ Replace $x_i \cdot x_j = \underbrace{K(x_i, x_j)}_{\text{Kernel funct}^n} = \phi(x_i) \cdot \phi(x_j)$

→ It finds the dot product in higher dimension

Ex. $K(x, y) = (1 + x_1 y_1 + x_2 y_2)^2$ (we don't need ϕ)

(*) Multiclass Classificat^n → One-against-all

→ " " one $(K(K-1)/2)$ classifier (K = class.

Use Majority Voting in classificat^n

(*) KNN does not build model from training data.

↳ It is parametric & weak learner

↳ SVM is strong learner bcoz once decision boundary is fixed, we are done

↳ decision tree is also weak learner

(*) Decision Tree Classifiers.

↳ A greedy divide & Conquer algo^m. Tree is constructed in a Top down recursive manner.

↳ Attributes are selected on the basis of an impurity funct^n (e.g. information gain).

↳ The objective is to reduce impurity (mixing of class labels) or uncertainty in data as much as possible

↳ A subset of data is pure if all instances belong to the same class.

↳ Attributes (based

↳ Entropy

↳ Entropy (

↓ Impurity Measure.

↳ 100 % +ve

↳ As data

↳ gain (D

↳ Handle (can be we can u

↳ Overfitti

↳ No classifier

↳ Complex Mo Simple

(*) Variance: of model or a value

⇒ Error = (Test)

↳ B

⇒ Ensemble

↳ The ensem classif

⇒ Non-linearly separable Prob^{n}s

↳ We allow "error" ξ_i in classificat^{n} based on o/p of discriminant fun. $w^T x + b$.

↳ approximates the no. of misclassified points

↳ New objective fun. $\frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i$ (C = tradeoff hyperparameter b/w error & margin

large C = high penalty for errors

s.t. $y_i(w x_i + b) \geq 1 - \xi_i$ & $\xi_i \geq 0$

↳ slack variable. Regularizat^{n} Parameter

Wolf-dual Prob^{n}

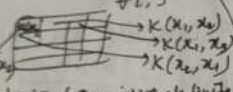
$$\text{Max } W(\alpha) = \sum_i \alpha_i - \frac{1}{2}\sum_i \alpha_i \alpha_j y_i y_j x_i \cdot x_j$$

s.t. $C \geq \alpha_i \geq 0$

$\sum \alpha_i y_i = 0$

⇒ Every semi-positive definite symmetric funct^{n} is a kernel (Mercer's Theorem)

↳ $K(x_i, x_j) = K(x_j, x_i)$ $\forall i, j$

$K =$ 

Gram Matrix (semi+ve definite symmetric)

(*) Polynomial kernel $= (x^T y + 1)^d$ (d = degree)

(*) RBF kernel $= \exp(-\|x - y\|^2 / 2\sigma^2)$ (σ = width)

(*) $K(x, y) = \tanh(k x^T y + \theta)$.

(*) Non-Linear SVMs (Kernel Machines)

↳ Ex. XOR prob^{m} can't be solved by LSVM (Linear SVM)

↳ Idea: The original input space can be mapped to some higher dimensional feature space $\phi(x)$ where data is separable.

↳ N data points are separable in N-1 dimensional

↳ Comput^{n} is higher to transform low dimension to higher.

↳ Kernel tricks solves this issue.

↳ Replace $x_i \cdot x_j = K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$

Kernel funct^{n}

→ It finds the dot product in higher dimension

Ex. $K(x, y) = (1 + x_1 y_1 + x_2 y_2)^2$ (we don't need ϕ)

(*) Multiclass classificat^{n}

→ One-against all

→ " " one $(K(K-1)/2)$ classifiers (K = class.

Use majority voting in classificat^{n}

(*) KNN does not build model from training data.

↳ It is parametric & weak learner

↳ SVM is strong learner bcoz one decision boundary is fixed, we are done

↳ decision tree is also weak learner

(*) Decision Tree Classifiers.

↳ A greedy divide & Conquer algo^{m}. Tree is constructed in a Top down recursive manner.

↳ Attributes are selected on the basis of an impurity funct^{n} (e.g. information gain).

↳ The objective is to reduce impurity (mixing of class labels) or uncertainty in data as much as possible

↳ A subset of data is pure if all instances belong to the same class.

↳ Attributes ar
(based on

↳ Entropy is

↳ Entropy (D)
↓
Impurity Measure.

↳ 100 % +ve &

↳ As data b

↳ gain (D, Ai

↳ Handle con
(can be m
we can use

↳ Overfitting

↳ No classifier i

↳ Complex Model
Simple "

(*) Variance: V
of model pr
or a value wh

⇒ Error = Bio
(Test)
↳ Bias

⇒ Ensemble ⇒

↳ The ensemble
classifier

↳ Attributes are chosen based on max^m info^m gain/gain ratio (based on info^m theory).

↳ Entropy is a measure of randomness.

↳ Entropy $(D) = -\sum_{J=1}^{|C|} \underbrace{Pr(C_J)}_{\text{Prob. of class } C_J \text{ is data set } D} \log_2 Pr(C_J)$ $\left[\sum_{J=1}^{|C|} Pr(C_J) = 1\right]$

↓ Impurity Measure.

↳ 100% +ve & 0% -ve has Entropy = 0 & 50% +ve & 50% -ve has (entropy = 1)

↳ As data becomes purer & purer entropy becomes smaller & (smaller)

↳ $gain(D, A_i) = entropy(D) - entropy_{A_i}(D)$

where, $entropy_{A_i}(D) = \underbrace{\sum_{J=1}^{v} \frac{|D_J|}{|D|} * entropy(D_J)}_{\text{weighted avg.}}$

↳ Handle continuous attribute by splitting into 2 intervals (can be more) at each node.
We can use mean value or info^m gain to find best threshold.

↳ Overfitting = Good accuracy on training data but poor on test data.

↳ No classifier is free from overfitting.

↳ Complex Model = low bias & high variance.
Simple " = high bias & low "

Bias = High Training & Test Error.
↳ It is the difference b/w the avg. prediction of our model & correct value which we are trying to predict

(*) Variance: Variance is the variability of model prediction for a given data point or a value which tells us spread of data.

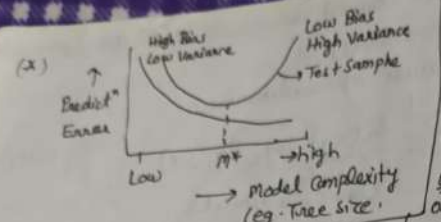
⇒ Error (Test) = Bias^2 + Variance + Irreducible Error (ε)

↳ Bias $= \left(E[\hat{f}(x)] - f(x)\right)$, Variance $= E\left[\left(f(x) - E[\hat{f}(x)]\right)^2\right]$

⇒ Ensemble ⇒ $D \begin{cases} D_1 \rightarrow C_1 \\ D_2 \rightarrow C_2 \\ D_3 \rightarrow C_3 \end{cases} \Rightarrow C^*$ (strong classifier)

(D_1, D_2, D_3 = data sets. C_1, C_2, C_3 = classifiers (weak))

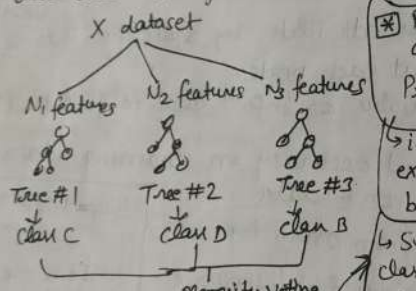
↳ The ensemble makes a wrong prediction if the majority of the classifiers makes a wrong prediction.



⊛ Random Forest Classifiers.

↳ It is an extension to bagging which uses de-correlated trees.

↳ Here we construct a decision tree on each bootstrapped sample by taking $\sqrt{p}$ features & then take the majority vote.



⊛ Bagging (Bootstrap aggregation)

↳ It averages a given procedure over many samples to reduce its variance.

↳ We use sampling with replacement.

Ex.

original data	1	2	3	4	5	6	7	8	9	10
Bagging (Round ①)	7	8	10	8	2	5	10	10	5	9
Bagging (Round ②)	1	4	9	1	2	3	2	7	3	2
Bagging (Round ③)	1	8	5	10	5	5	9	6	3	7

⊛ For each set, the prob. that a given example, appears in it

$$P_r(x \in D_i) = 1 - \left(1 - \frac{1}{n}\right)^n \Rightarrow (1)$$

↳ i.e. less than $\frac{2}{3}$ of the examples appear in one bootstrap sample.

⊛ Boosting

↳ Also works by manipulating training set but classifiers are trained sequentially.

↳ Each classifier is trained by given knowledge of the performance of previously trained classifiers — focus on hard examples.

↳ Final Classifier :- weighted sum of component classifiers.

↳ In Random Forest, each training example is not provided with weights but here weights are provided.

↳ It makes weak learner, stronger.

↳ Suppose we have a binary classificat^n prob^m. $\{x_i, y_i\}$ where $y_i \in \{+1, -1\}$ & we have to make a dream team of classifiers. i.e. $K_i(x_i) \in \{-1, +1\}$

↳ Suppose we have l classifiers

$$C(x_i) = \alpha_1 K_1(x_i) + \alpha_2 K_2(x_i) + \dots + \alpha_l K_l(x_i)$$

& predict^n $\hat{y}_i = \text{Sign}(C(x_i))$

↳ Ex. Adaboost (Adaptive Boosting)

↳ We provide more importance to those classifiers which misclassify points.

↳ Initially, all data points have equal weights. & after each iterat^n, weights of each data point changed.

→ Boosting have 3 pa

↳ If classifier is su

" " "

↳ Suppose, we hav

assigns 0 if the

& this matrix is

	1	2	3	- -	L
x_1					
x_2					
⋮					
x_N					

→ In weighting, we have to determine the proper weight α_m by mini E as a funct^n of α_m

i.e. $\frac{dE}{d\alpha_m} = 0$

⊛ Dimensionality Reduction

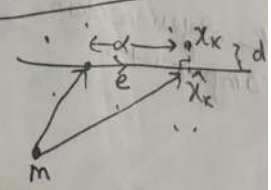
PCA

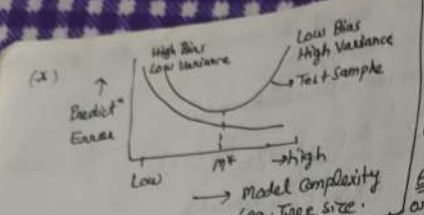
↳ Given a large no. o features d, we want end up with a smal no. of features p,

↳ The reduced set should have as mu

↳ If the features c

& Def

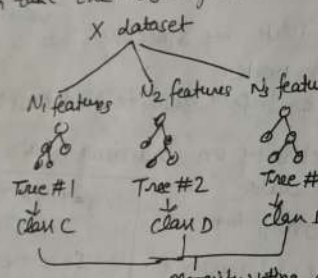




Random Forest Classifiers.

↳ It is an extension to bagging which uses de-correlated trees.

↳ Here we construct a decision tree on each bootstrapped sample by taking $\sqrt{p}$ features & then take the majority vote.



Bagging (Bootstrap aggregation)

↳ It averages a given procedure over many samples to reduce its variance.

↳ We use sampling with replacement.

Ex.

original data	1	2	3	4	5	6	7	8	9	10
Bagging (Round ①)	7	8	10	8	2	5	10	10	5	10
Bagging (Round ②)	1	4	9	1	2	3	2	7	3	2
Bagging (Round ③)	1	8	5	10	5	5	9	6	3	7

For each set, the prob. that a given example, appears in it

$$Pr(x \in D_i) = 1 - \left(1 - \frac{1}{n}\right)^n \Rightarrow 63\%$$

↳ i.e. less than $\frac{2}{3}$ of the examples appear in one bootstrap sample.

Boosting

↳ Also works by manipulating training set but classifiers are trained sequentially.

↳ Each classifier is trained by given knowledge of the performance of previously trained classifiers — focus on hard examples.

↳ Final Classifier :- weighted sum of component classifiers.

↳ In Random Forest, each training example is not provided with weights but here weights are provided.

↳ It makes weak learner, stronger.

↳ Suppose we have a binary classificat^n prob^m. $\{x_i, y_i\}$ where $y_i \in \{+1, -1\}$ & we have to make a dream team of classifiers. i.e. $K_i(x_i) \in \{-1, +1\}$

↳ Suppose we have 11 classifi

$$C(x_i) = \alpha_1 K_1(x_i) + \alpha_2 K_2(x_i) + \dots + \alpha_{11} K_{11}(x_i)$$

& predict^n $\hat{y}_i = \text{Sign}(C(x_i))$

↳ Ex. Adaboost (Adaptive Boosting)

↳ We provide more importance to those classifiers which misclassify points

↳ Initially, all data points have equal weights. & after each iterat^n, weights of each data point changed.

→ Boosting have 3 parts →

↳ If classifier is success

" " " fail

↳ Suppose we have

assigns 0 if there i

& this matrix is upd

	1	2	3	...	C
x_1					
x_2					
⋮					
x_N					

→ In weighting, we have to determine the proper weight α_m by minimizing E as a funct^n of α_m.

i.e. $\frac{dE}{d\alpha_m} = 0$

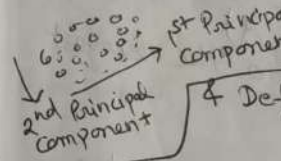
Dimensionality Reduction

PCA

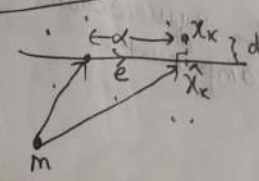
↳ Given a large no. of features d, we want to end up with a smaller no. of features p, p<<

↳ The reduced set of fe should have as much

↳ If the features are



& Define



→ Boosting have 3 parts → ① Scouting ② Drafting ③ Weight Update

↳ If classifier is successful each time ⇒ Assign cost $e^{-\beta}$, $\beta > 0$

" " " failure then assign cost, e^{β}, $\beta > 0$.

↳ suppose we have L classifiers then scouting matrix assigns 0 if there is a success & 1 if there is a failure & this matrix is updated each time

	1	2	3	- -	L
x_1					
x_2					
⋮					
x_N					

In Drafting.

at $m-1^{th}$ iteration $C_{m-1}(x_i) = \alpha_1 K_1(x_i) + \alpha_2 K_2(x_i) + \dots + \alpha_{m-1} K_{m-1}(x_i)$

& at m^{th} iteration, $C_m(x_i) = C_{m-1}(x_i) + \alpha_m K_m(x_i)$

→ Exponential loss of total classifier.

$$E = \sum_{i=1}^{N} e^{-y_i \frac{(C_m(x_i))}{\beta}}, \quad y_i \in \{-1, +1\}$$

→ In weighting, we have to determine the proper weight α_m by minimizing E as a funct^n of α_m.

i.e. $\frac{dE}{d\alpha_m} = 0$

Stacking.

↳ Learn a funct^n that combines the predictions of the individual classifiers.

Attributes	C_1	C_2	- -	C_n	class
x_{11} x_{12} - x_{1m}	t	t			t
x_{21} - -	f	t			t
-					⋮

feature vector for meta classifier

meta classifier (may be decision tree)

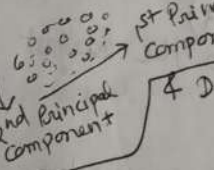
Dimensionality Reduction

PCA

↳ Given a large no. of features d, we want to end up with a smaller no. of features p, $p << d$

↳ The reduced set of features should have as much of info^m as possible from the original set of features.

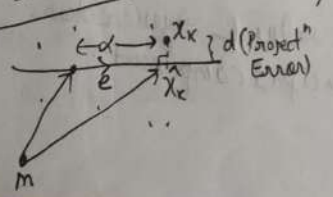
↳ If the features are correlated, we should extract a set of uncorrelated features.



⇒ Derivation of PCA

Let, m (centre of mass) $= \frac{1}{n} \sum_{k=1}^{n} x_k$

& Define a squared loss funct^n as $J_0(x_0) = \sum_{k=1}^{n} \| x_0 - x_k \|^2$



→ $\hat{x}_k = m + \alpha e$

Now, Project^n error (d) $= \| x_k - \hat{x}_k \|_2^2$ should be minimized

$= \| (m + \alpha_k e) - x_k \|_2^2$

This is for a single point x_k

→ For a set of multivariate points -

$$J_1(\alpha_1, \alpha_2, \ldots, \alpha_n, e) = \sum_{k=1}^{n} \| m + \alpha_k e - x_k \|^2$$

$$= \sum_{k=1}^{n} \| \alpha_k e - (x_k - m) \|^2$$

$$= \sum_{k=1}^{n} \alpha_k^2 \|e\|^2 - 2 \sum_{k=1}^{n} \alpha_k e^T (x_k - m) + \sum_{k=1}^{n} \|x_k - m\|^2$$

Now, let, $\|e\| = 1$, differentiating wrt α_k & setting it to zero,

$\boxed{\alpha_k = e^T (x_k - m)}$ So, $J_1(e) = \sum_{k=1}^{n} \alpha_k^2 - 2 \sum_{k=1}^{n} \alpha_k^2 + \sum_{k=1}^{n} \|x_k - m\|^2$

↳ Let's define the scatter matrix $S = \sum_{k=1}^{n} (x_k - m)(x_k - m)^T$

$$J_1(e) = -\sum_{k=1}^{n} \left(e^T (x_k - m)\right)^2 + \sum_{k=1}^{n} \|x_k - m\|^2$$

$$= -\sum_{k=1}^{n} e^T (x_k - m) \left[e^T (x_k - m)\right]^T + \sum_{k=1}^{n} \|x_k - m\|^2$$

$$= -\sum_{k=1}^{n} e^T \underbrace{(x_k - m)(x_k - m)^T}_{S} e + \sum_{k=1}^{n} \|x_k - m\|^2$$

So, $\overset{\text{minimize}}{J_1(e)} = -e^T S e$ + const wrt e & α

so, maximize $e^T S e$ such that $\|e\| = 1$ or $e^T e = 1$

So, $u = e^T S e - \lambda (e^T e - 1) \Rightarrow \frac{\partial u}{\partial e} = 0 \Rightarrow Se = \lambda e$

So, $e^T S e = e^T \lambda e = \lambda e^T e = \lambda$ (So, maximizing $e^T S e$ is same as maximizing λ)

↳ So, we project along eigenvector corresponding to the largest eigenvalue of matrix S.

↳ Scatter Matrix $(S) = XX^T$

dataset matrix → Find eigen value & eigen vector to get principal component.

(*) Image pixels a
10,000 dimension

(*) XX^T & $X^T X$ h

Proof $XX^T y =$

↳ Instead of c

(*) Preserving
lead to qua

(*) The direct^n o

(*) A more di

*) Fisher Linear Disc

(*) PCA preserves

(*) Find project^n

(*) PCA finds th
a lower d
of max^m
may be u

(*) Consider pr
a unit vecto
from the or

↳ Let $\tilde{\mu}_1$ & $\tilde{\mu}_2$
μ_1 & μ_2 b

Then separat^n b/w

The prob^m with
of the class

(*) Image pixels are lowest level features. For 100x100 image = 10,000 dimensions ⇒ slow & lots of storage. ⇒ Dimensionality Red^n.

(*) XX^T & X^TX have the same eigenvalues.

Proof. $XX^Ty = \lambda y \Rightarrow X^T(XX^Ty) = \lambda X^Ty \Rightarrow (X^TX)\underbrace{(X^Ty)}_{\text{vector}} = \lambda \underbrace{(X^Ty)}_{\text{vector}}$

↳ Instead of calculating XX^T, we calculate X^TX.

(*) Preserving variance (minimizing MSE) does not necessarily lead to qualitatively good reconstruction.

(*) The direct^n of max^m variance is not always good for classificat^n.

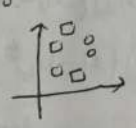
(*) A more discriminative subspace : FLD. (Fischer Linear Discriminants) (Fischer Faces)

*) Fisher Linear Discriminants → "Fischer Faces"

(*) PCA preserves max^m variance & FLD preserves discrimination.

(*) Find project^n that maximizes scatter b/w classes & minimizes scatter ~~within~~ classes

(*) PCA finds the most accurate data representation in a lower dimensional space & projects data in the direction of max^m variance. However the directions of max^m variance may be useless for classificat^n.



$\xrightarrow{PCA}$ (Non separable.

↳ FLD project data to a line which preserves direction useful for data classification.

(*) Consider projection on a line. Let the line direction be given by a unit vector v. Scalar v^tx_i is the distance of project^n of x_i from the origin. Thus v^tx_i is the project^n of x_i into a 1D subspace.

Let $\tilde{\mu}_1$ & $\tilde{\mu}_2$ be the means of projections of classes ① & ②. &

μ_1 & μ_2 be the means of ~~pro~~ class ① & ②.

Then seperat^n b/w project^{n}s of diff. classes $= |\tilde{\mu}_1 - \tilde{\mu}_2| = \left|\frac{1}{n_1}\sum_{x_i \in C_1}^{n_1} v^tx_i - \frac{1}{n_2}\sum_{x_i \in C_2}^{n_2} v^tx_i\right|$

$= v^t\mu_1 - v^t\mu_2$

The prob^m with $|\tilde{\mu}_1 - \tilde{\mu}_2|$ is that it does not consider the variance of the classes.

(*) Image pixels are lowest level features. For 100×100 image = 10,000 dimensions ⇒ Slow & lots of storage. ⇒ Dimensionality Red^n.

(*) XX^T & X^TX have the same eigenvalues.

Proof $XX^Ty = \lambda y \Rightarrow X^T(XX^Ty) = \lambda X^Ty \Rightarrow (X^TX)\underbrace{(X^Ty)}_{\text{vector}} = \lambda \underbrace{(X^Ty)}_{\text{vector}}$.

↳ Instead of calculating XX^T, we calculate X^TX.

(*) Preserving variance (minimizing MSE) does not necessarily lead to qualitatively good reconstruction.

(*) The direct^n of max^m variance is not always good for classificat^n.

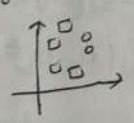
(*) A more discriminative subspace : FLD. (Fischer Linear Discriminants) (Fischer Faces)

*) Fisher Linear Discriminants → "Fischer Faces"

(*) PCA preserves max^m variance & FLD preserves discrimination.

(*) Find project^n that maximizes scatter b/w classes & minimizes scatter within classes.

(*) PCA finds the most accurate data representation in a lower dimensional space & projects data in the direction of max^m variance. However the directions of max^m variance may be useless for classificat^n.



PCA ⇒ Non separable.

↳ FLD project data to a line which preserves direction useful for data classification.

(*) Consider projection on a line. Let the line direction be given by a unit vector v. Scalar $v^t x_i$ is the distance of project^n of x_i from the origin. Thus $v^t x_i$ is the project^n of x_i into a 1D subspace.

↳ Let $\tilde{\mu}_1$ & $\tilde{\mu}_2$ be the means of projection of classes ① & ②. & μ_1 & μ_2 be the means of class ① & ②.

Then seperat^n b/w project^n's of diff. classes $= |\tilde{\mu}_1 - \tilde{\mu}_2| = \left|\frac{1}{n_1}\sum_{x_i \in C_1} v^t x_i - \frac{1}{n_2}\sum_{x_i \in C_2} v^t x_i\right|$

$= v^t \mu_1 - v^t \mu_2$

The prob^m with $|\tilde{\mu}_1 - \tilde{\mu}_2|$ is that it does not consider the variance of the classes.

(*) Image pixels are lowest level features. For 100x100 image = 10,000 dimensions ⇒ slow & lots of storage. ⇒ Dimensionality Red^n.

(*) XX^T & X^TX have the same eigenvalues.

Proof $XX^Ty = \lambda y \Rightarrow X^T(XX^Ty) = \lambda X^Ty \Rightarrow (X^TX)\underbrace{(X^Ty)}_{vector} = \lambda \underbrace{(X^Ty)}_{vector}$.

↳ Instead of calculating XX^T, we calculate X^TX.

(*) Preserving variance (minimizing MSE) does not necessarily lead to qualitatively good reconstruction.

(*) The direct^n of max^m variance is not always good for classificat^n.

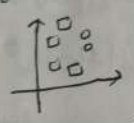
(*) A more discriminative subspace : FLD. (Fischer Linear Discriminants) (Fischer Faces)

*) Fisher Linear Discriminants → "Fischer Faces"

(*) PCA preserves max^m variance & FLD preserves discrimination.

(*) Find project^n that maximizes scatter b/w classes & minimizes scatter within classes

(*) PCA finds the most accurate data representation in a lower dimensional space & projects data in the direction of max^m variance. However the directions of max^m variance may be useless for classificat^n.



PCA ⟹ Non separable.

↳ FLD project data to a line which preserves direction useful for data classification.

(*) Consider projection on a line. Let the line direction be given by a unit vector v. Scalar v^tx_i is the distance of project^n of x_i from the origin. Thus v^tx_i is the project^n of x_i into a 1D subspace.

↳ Let $\tilde{\mu}_1$ & $\tilde{\mu}_2$ be the means of projection of classes ① & ②. &

μ_1 & μ_2 be the means of class ① & ②.

Then seperat^n b/w project^n's of diff. classes $= |\tilde{\mu}_1 - \tilde{\mu}_2| = \left|\frac{1}{n_1}\sum_{x_i \in C_1}^{n_1} v^tx_i - \frac{1}{n_2}\sum_{x_i \in C_2}^{n_2} v^tx_i\right|$

$= v^t\mu_1 - v^t\mu_2$

The prob^m with $|\tilde{\mu}_1 - \tilde{\mu}_2|$ is that it does not consider the variance of the classes.

So, we need to normalize $|\tilde{\mu}_1 - \tilde{\mu}_2|$ by a factor which is proportional to variance.

→ Scatter $S = \sum_{i=1}^{n} (z_i - \mu_z)^2$ ↳ Scatter is just sample variance multiplied by n. & it measure the same thing as variance (the spread of data around the mean), it is on different scale than variance.

→ So, let $y_i = v^t x_i$, (v = projection matrix)

So, $\tilde{s}_1^2 = \sum_{y_i \in C_1} (y_i - \tilde{\mu}_1)^2$

& $\tilde{s}_2^2 = \sum_{y_i \in C_2} (y_i - \tilde{\mu}_2)^2$

↳ After Normalizat^n, maximize $\boxed{J(v) = \frac{(\tilde{\mu}_1 - \tilde{\mu}_2)^2}{\tilde{s}_1^2 + \tilde{s}_2^2}}$ → maximize → minimize

⇒ Class within Scatter Matrix $S_w = S_1 + S_2 = \underbrace{\sum_{x_i \in C_1} (x_i - \mu_1)(x_i - \mu_1)^t}_{S_1} + \underbrace{\sum_{x_i \in C_2} (x_i - \mu_2)(x_i - \mu_2)^t}_{S_2}$

& $\tilde{s}_1^2 = \sum_{y_i \in C_1} (v^t x_i - v^t \mu_1)^2$

$= v^t S_1 v$ & $\tilde{s}_2^2 = v^t S_2 v$

So, $\boxed{\tilde{s}_1^2 + \tilde{s}_2^2 = v^t (S_1 + S_2) v = v^t S_w v}$

⇒ Between the Class Scatter Matrix: $S_B = (\mu_1 - \mu_2)(\mu_1 - \mu_2)^t$

& $(\tilde{\mu}_1 - \tilde{\mu}_2)^2 = (v^t \mu_1 - v^t \mu_2)^2 = v^t (\mu_1 - \mu_2)(\mu_1 - \mu_2)^t v$

⇒ $\boxed{(\tilde{\mu}_1 - \tilde{\mu}_2)^2 = v^t S_B v}$

(*) So, $\boxed{J(v) = \frac{v^t S_B v}{v^t S_w v}} \Rightarrow \frac{d}{dv} J(v) = 0 \Rightarrow \boxed{S_B v = \lambda S_w v}$ → Generalized Eigenvalue Probl^m.

$\lambda = \frac{v^t S_B v}{v^t S_w v}$

So, $\boxed{S_w^{-1} S_B v = \lambda v}$ (S_w^{-1} should be existed (full rank))

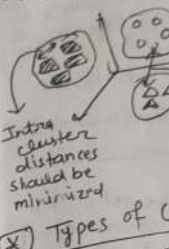
$\boxed{J(v) = 0 \text{ if } \mu_1 = \mu_2}$

$\text{rank}(AB) \le \min(\text{rank}(A), \text{rank}(B))$

(*) Clustering / Cluster Analy

→ Clustering is a big unsa

→ Objective is to detect n



(*) Types of clustering

① Partitional Algo's.

② Hierarchical " (eg. Dendogra

No need to specify no. of clusters.

Top-down (divisive)

Starting with all data in a single cluster, consider every possible way to divide the cluster into two. Choose the best division & recurse

Bottom-Up (agglomerative)

↳ start with each item in its own cluste & find the b pair to me into a new Repeat u clusters to get

Partitioning Clustering

↳ Each instance is placed

↳ Specify no. of clusters

⇒ K-Means Clustering. →

↳ Hierarchical clustering al algo^m typically have gl

↳ Cluster Membership Matrix.

$U = \begin{array}{c|cc} & x_1 & x_2 \\ \hline c_1 & 1 & 0 \\ c_2 & 0 & 1 \\ c_3 & & \end{array}$

$U^T U$ = Cardinality of Cluster

Need to Minimize (Local m (Lloy